INTERVIEW PREPARATION GUIDE

100

Node.js Interview Questions

In Simple, Easy English & Short Answers

Prepared by **Pushkar Singh**

August 2026

A QUICK-REFERENCE INTERVIEW COMPANION

CONTENTS

Table of Contents

01	Node.js Basics	**3**
02	Modules & npm	**5**
03	Core Modules	**7**
04	Asynchronous Programming	**9**
05	Express.js	**11**
06	Databases & Node.js	**13**
07	Error Handling & Debugging	**15**
08	Streams & Buffers	**17**
09	Security & Middleware	**19**
10	Testing, Deployment & Miscellaneous	**21**

Ten questions per chapter — 100 questions in all

CHAPTER 01

Node.js Basics

001 What is Node.js?

Node.js is a runtime that lets us run JavaScript outside the browser, mostly on the server side. It is built on Chrome's V8 engine. It is used to build fast and scalable server-side apps.

002 Why do we use Node.js?

We use Node.js because it is fast, lightweight, and good at handling many requests at the same time. It uses a single thread with non-blocking I/O. This makes it good for real-time apps like chat apps.

003 What is the difference between Node.js and JavaScript in the browser?

JavaScript in the browser runs in a browser environment and can access the DOM and window object. Node.js runs JavaScript on the server and does not have DOM access. It gives extra features like file system and networking.

004 What is npm?

npm stands for Node Package Manager. It is used to install, share, and manage packages (libraries) in Node.js projects. It comes automatically when we install Node.js.

005 What is the event loop in Node.js?

The event loop is what makes Node.js non-blocking. It checks the call stack and task queue again and again, and runs tasks one by one. This helps Node.js handle many operations without waiting.

006 What is single-threaded architecture in Node.js?

Node.js runs our JavaScript code on a single main thread. But it uses background threads (via libuv) for heavy tasks like file reading. This way, the main thread does not get blocked.

007 What is non-blocking I/O?

Non-blocking I/O means Node.js does not wait for one task, like reading a file, to finish before starting the next one. It moves to the next task and comes back later using a callback.

008 What is a callback function in Node.js?

A callback function is a function passed as an argument to another function, which runs later after some task is done. It is commonly used for handling things like file reading or API calls.

009 What is REPL in Node.js?

REPL stands for Read-Eval-Print-Loop. It is a simple command-line tool that comes with Node.js where we can write and test JavaScript code line by line and see the output right away.

010 What is the difference between Node.js and a framework like Express.js?

Node.js is a runtime environment for running JavaScript on the server. Express.js is a framework built on top of Node.js that makes it easier to build web apps and APIs with less code.

CHAPTER 02

Modules & npm

011 What is a module in Node.js?

A module is a separate file that has its own code, like functions or variables, which we can use in other files. It helps in organizing code into small, reusable parts.

012 What are the types of modules in Node.js?

There are three types: core modules (built-in, like fs and http), local modules (files we create ourselves), and third-party modules (installed using npm, like express).

013 What is require() used for?

require() is used to import a module into our file so we can use its functions or variables. It is the older, CommonJS way of importing modules in Node.js.

014 What is module.exports?

module.exports is used to export functions, objects, or variables from one file so they can be used in another file using require(). It is how we share code between files.

015 What is the difference between require and import?

require is the CommonJS way of importing modules, used by default in Node.js. import is the newer ES6 way, which also works in Node.js if we set the type to "module" in package.json.

016 What is package.json?

package.json is a file that stores information about a Node.js project, like its name, version, dependencies, and scripts. It helps npm know what packages to install.

017 What is package-lock.json?

package-lock.json keeps a record of the exact versions of every package installed in a project. It makes sure everyone working on the project gets the same versions, avoiding version problems.

018 What is the difference between dependencies and devDependencies?

dependencies are packages needed to run the app in production, like express. devDependencies are packages needed only during development, like testing tools, and are not required in production.

019 How do you install a package globally vs locally in npm?

A local install (npm install package-name) adds the package only to the current project folder. A global install (npm install -g package-name) makes the package available everywhere on the system.

020 What is npx?

npx is a tool that comes with npm and lets us run a package without installing it permanently. It is useful for running one-time commands, like creating a new project.

CHAPTER 03

Core Modules

021 What is the fs module?

The fs (file system) module is a built-in Node.js module used to work with files, like reading, writing, updating, and deleting files on the computer.

022 What is the difference between fs.readFile and fs.readFileSync?

fs.readFile is asynchronous and does not block other code while reading a file. fs.readFileSync is synchronous and stops the program until the file is fully read.

023 What is the http module used for?

The http module is a built-in module in Node.js used to create web servers and handle HTTP requests and responses, without needing any extra framework.

024 What is the path module?

The path module is a built-in module that helps us work with file and folder paths easily, like joining paths or getting the file extension, on any operating system.

025 What is the os module in Node.js?

The os module is a built-in module that gives information about the computer's operating system, like CPU, memory, and platform details.

026 What is the events module in Node.js?

The events module lets us create and handle our own events in code using the EventEmitter class. We can emit an event and listen for it somewhere else in the code.

027 What is EventEmitter in Node.js?

EventEmitter is a class from the events module that lets objects emit named events, so other parts of the code can listen and react to them using .on() and .emit().

028 What is the url module used for?

The url module is a built-in module that helps us work with URLs, like breaking a URL into parts (host, path, query) or building a new URL.

029 What is process in Node.js?

process is a global object in Node.js that gives information about the current running program, like command-line arguments, environment variables, and a way to exit the program.

030 What is the difference between process.nextTick() and setImmediate()?

process.nextTick() runs its callback right after the current operation finishes, before the event loop continues. setImmediate() runs its callback in the next round of the event loop.

CHAPTER 04

Asynchronous Programming

031 What is synchronous vs asynchronous programming?

In synchronous programming, each line of code runs one after another, waiting for the previous one to finish. In asynchronous programming, tasks can run in the background without making the code wait.

032 What is a Promise in Node.js?

A Promise is an object that represents a value which may be available now, later, or never. It helps us handle asynchronous tasks in a cleaner way, using .then() and .catch().

033 What are the states of a Promise?

A Promise has three states: pending (task not finished yet), fulfilled (task completed successfully), and rejected (task failed with an error).

034 What is async/await?

async/await is a simpler way to write asynchronous code that looks like normal step-by-step code. async makes a function return a Promise, and await pauses the function until the Promise is done.

035 What is callback hell?

Callback hell happens when we write many callbacks inside each other, making the code look messy and hard to read, like a staircase going down. Promises and async/await help avoid this.

036 What is the difference between Promise.all and Promise.race?

Promise.all waits for all given promises to finish and returns their results together. Promise.race returns the result of whichever promise finishes first, ignoring the others.

037 How do you handle errors in async/await?

We handle errors in async/await using a try...catch block. We put the code that might fail inside try, and if an error happens, it is caught in the catch block.

038 What is the microtask queue in Node.js?

The microtask queue holds tasks like Promise callbacks, which run right after the current code finishes, before the event loop moves to the next task like setTimeout.

039 What is setTimeout used for in Node.js?

setTimeout is used to run a function after a certain amount of time (in milliseconds) has passed. It does not stop other code from running while it waits.

040 What is the difference between setTimeout and setInterval?

setTimeout runs a function only once after a delay. setInterval runs a function again and again after every fixed delay, until it is stopped using clearInterval.

CHAPTER 05

Express.js

041 What is Express.js?

Express.js is a simple and popular framework built on top of Node.js. It makes it easy to create web servers, APIs, and handle routes with less code.

042 What is middleware in Express.js?

Middleware is a function that runs between the request and the response in Express. It can check, change, or log the request before it reaches the final route handler.

043 How do you create a basic route in Express?

We create a route using app.get(), app.post(), and so on, by giving a path and a function, like app.get('/home', (req, res) => { res.send('Hello') }).

044 What is the difference between app.use() and app.get()?

app.use() adds middleware that runs for all types of requests matching the path. app.get() runs code only for GET requests on that specific route.

045 What are route parameters in Express?

Route parameters are dynamic parts of a URL, written with a colon, like /user/:id. We can access their value using req.params inside the route handler.

046 What is the difference between req.params, req.query, and req.body?

req.params gets values from the URL path (like /user/:id). req.query gets values from the URL's query string (like ?name=John). req.body gets data sent in the request body.

047 What is CORS and why is it needed?

CORS (Cross-Origin Resource Sharing) is a security feature that controls which websites can call your API from a different domain. We use the cors package in Express to allow this safely.

048 What is body-parser in Express?

body-parser is a middleware that reads the data sent in the body of a request, like JSON or form data, and makes it available in req.body. Express now has this built in as express.json().

049 How do you handle errors in Express?

We handle errors in Express using an error-handling middleware function, which has four arguments (err, req, res, next), placed after all the other routes.

050 What is routing in Express.js?

Routing means deciding what code should run when a specific URL and method, like GET or POST, is requested. Express lets us define different routes for different URLs easily.

CHAPTER 06

Databases & Node.js

051 How does Node.js connect to a database like MongoDB?

Node.js connects to MongoDB using a package like mongoose or the official MongoDB driver. We write connection code with the database URL, and then we can read and write data.

052 What is Mongoose?

Mongoose is a popular npm package (an ODM, or Object Data Modeling library) used to connect Node.js apps with MongoDB. It helps us define schemas and makes working with data easier.

053 What is a schema in Mongoose?

A schema in Mongoose defines the structure of a document, like what fields it has and their data types. For example, a user schema might have name (String) and age (Number).

054 How do you connect Node.js to a SQL database like MySQL?

We connect Node.js to MySQL using a package like mysql2 or sequelize. We give the database details, like host, user, and password, and then we can run SQL queries from our code.

055 What is an ORM?

ORM stands for Object Relational Mapping. It lets us work with a SQL database using JavaScript objects and methods instead of writing raw SQL queries. Sequelize is a popular ORM for Node.js.

056 What is connection pooling in databases?

Connection pooling means keeping a group (pool) of database connections ready to use, instead of opening a new connection every time. This makes database work faster and more efficient.

057 What is CRUD?

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations we do with data in almost every application and database.

058 How do you handle database errors in Node.js?

We handle database errors using try...catch blocks with async/await, or .catch() with promises, to catch any errors during database work and respond properly instead of crashing.

059 What is a NoSQL database?

A NoSQL database stores data in a flexible way, like documents or key-value pairs, instead of tables and rows like SQL databases. MongoDB is a popular example of a NoSQL database.

060 What is indexing in a database and why is it useful?

Indexing creates a special data structure that helps the database find data faster, similar to an index in a book. It speeds up search queries but can slow down write operations a little.

CHAPTER 07

Error Handling & Debugging

061 How do you handle errors in Node.js?

We handle errors using try...catch blocks for normal and async/await code, and using .catch() for promises. For unexpected errors, we can also use process-level error handlers.

062 What is the try-catch block used for?

The try-catch block is used to catch errors that happen while running code. We put the risky code inside try, and if an error occurs, it is handled inside catch instead of crashing the app.

063 What is an uncaught exception in Node.js?

An uncaught exception is an error that is not handled anywhere in the code using try-catch. If not handled, it can crash the whole Node.js application.

064 How do you debug a Node.js application?

We can debug Node.js apps using console.log() statements, the built-in Node.js debugger (node inspect), or tools like Chrome DevTools and VS Code's debugger for a better experience.

065 What is the purpose of the debugger keyword in Node.js?

The debugger keyword is used in code to create a breakpoint. When we run the code in debug mode, it pauses at that line, letting us check variable values step by step.

066 What is a custom error class in Node.js?

A custom error class is a class we create by extending the built-in Error class, to make our own error types with extra details, like a status code, for better error handling.

067 What is the difference between operational errors and programmer errors?

Operational errors are expected problems, like a failed API call or wrong user input, which we should handle nicely. Programmer errors are bugs in the code, like typos, which need to be fixed.

068 How do you log errors in a Node.js application?

We can log errors using console.error() for simple cases, or use logging libraries like winston or morgan for more detailed and organized logs, especially in production apps.

069 What is a stack trace?

A stack trace shows the path of function calls that led to an error, helping us find exactly where and why the error happened in the code.

070 How can you prevent your Node.js app from crashing due to unhandled errors?

We can use try-catch blocks properly, add global handlers for uncaughtException and unhandledRejection events, and use a process manager like PM2 to restart the app if it crashes.

CHAPTER 08

Streams & Buffers

071 What is a stream in Node.js?

A stream is a way to handle reading or writing data in small pieces (chunks) instead of loading everything into memory at once. It is useful for working with large files or data.

072 What are the types of streams in Node.js?

There are four types: Readable (for reading data), Writable (for writing data), Duplex (both reading and writing), and Transform (a duplex stream that can also change the data).

073 What is a buffer in Node.js?

A buffer is a temporary storage area used to hold raw binary data, like the data being read from a file or network, before it is fully processed.

074 Why do we use streams instead of loading the whole file into memory?

We use streams because they save memory. Instead of loading a big file completely, streams process small chunks at a time, which is more efficient for large files.

075 What is piping in Node.js streams?

Piping means connecting the output of one stream directly to the input of another stream using the .pipe() method, like reading a file and writing it somewhere else in one step.

076 What is the difference between Buffer and Stream?

A Buffer holds a fixed amount of binary data in memory all at once. A Stream handles data as a continuous flow of small chunks, which is better for large amounts of data.

077 What is backpressure in streams?

Backpressure happens when data is coming in faster than it can be processed or written out. Node.js streams handle backpressure automatically to avoid using too much memory.

078 How do you create a readable stream in Node.js?

We can create a readable stream using the fs module, like fs.createReadStream('file.txt'), which lets us read a file bit by bit instead of all at once.

079 What is the difference between Buffer.alloc() and Buffer.from()?

Buffer.alloc() creates a new empty buffer of a given size, filled with zeros. Buffer.from() creates a buffer from existing data, like a string or an array.

080 Where are streams commonly used in real applications?

Streams are commonly used for reading and writing large files, handling video or audio data, processing HTTP requests and responses, and compressing or decompressing data.

CHAPTER 09

Security & Middleware

081 What is middleware in Node.js/Express?

Middleware is a function that has access to the request, response, and the next function. It runs in between the request coming in and the response going out.

082 What is helmet in Express.js?

helmet is a middleware package that helps make Express apps more secure by setting some important HTTP headers, protecting against common attacks like clickjacking.

083 How do you protect a Node.js app from SQL injection?

We protect against SQL injection by using parameterized queries or ORMs, like Sequelize, instead of directly putting user input into SQL queries as plain text.

084 What is JWT (JSON Web Token)?

JWT is a compact, secure way to send information between two parties as a token. It is commonly used in Node.js apps for login, letting the server check a user without checking the database every time.

085 How do you hash passwords in Node.js?

We hash passwords using libraries like bcrypt, which turns a plain password into a scrambled string that is very hard to reverse, so we never store plain text passwords.

086 What is rate limiting and why is it used?

Rate limiting controls how many requests a user or IP address can make in a given time. It is used to protect the server from being overloaded or attacked, like a brute-force attack.

087 What is an environment variable and why do we use .env files?

An environment variable stores setting values, like API keys or database URLs, outside the code. We use .env files with a package like dotenv to keep these values secret and separate from the code.

088 What is CSRF and how can Node.js apps prevent it?

CSRF (Cross-Site Request Forgery) is an attack where a bad website tricks a user's browser into making unwanted requests to another site. Node.js apps prevent it using CSRF tokens and safe cookie settings.

089 Why should you validate user input in a Node.js app?

We should validate user input to make sure the data is correct and safe before using it, which helps prevent bugs, crashes, and security attacks like injection attacks.

090 What is the purpose of using HTTPS in a Node.js application?

HTTPS encrypts the data sent between the client and server, protecting sensitive information like passwords and personal data from being read by attackers.

CHAPTER 10

Testing, Deployment & Miscellaneous

091 What testing frameworks are commonly used with Node.js?

Common testing frameworks include Jest, Mocha, and Jasmine. They are used to write and run tests to check if our Node.js code works correctly.

092 What is unit testing in Node.js?

Unit testing means testing small, individual pieces of code, like a single function, to make sure each piece works correctly on its own, separate from the rest of the app.

093 What is the difference between unit testing and integration testing?

Unit testing checks small individual parts of code in isolation. Integration testing checks how different parts of the app work together, like a route talking to a database.

094 What is PM2 in Node.js?

PM2 is a process manager for Node.js applications. It helps keep an app running all the time, restarts it automatically if it crashes, and can manage multiple copies of an app.

095 What is clustering in Node.js?

Clustering lets a Node.js app use multiple CPU cores by creating several copies (worker processes) of the same app, which helps handle more traffic at the same time.

096 What is the difference between development and production environments?

The development environment is used while building and testing the app, often with extra logging and debugging tools. The production environment is the live version used by real users.

097 How do you deploy a Node.js application?

We can deploy a Node.js app on cloud platforms like AWS, Heroku, or DigitalOcean, or use containers like Docker. This usually includes setting environment variables and starting the app with a process manager.

098 What is a RESTful API?

A RESTful API is a way of building web services that follow REST rules, using standard HTTP methods like GET, POST, PUT, and DELETE to work with resources through URLs.

099 What is the difference between Node.js and Deno?

Node.js and Deno are both JavaScript runtimes, but Deno is newer, has better built-in security, and supports TypeScript directly, while Node.js has a much bigger existing collection of packages.

100 What is microservices architecture in the context of Node.js?

Microservices architecture means building an application as many small, independent services instead of one big app. Node.js is popular for this because it is lightweight and fast for handling many small services.